

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A Algorithm Design Challenge

Code of Conduct:

I G.Karthik Reddy-192525047 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: G.Karthik Reddy

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Q1. Greedy Best-First Search

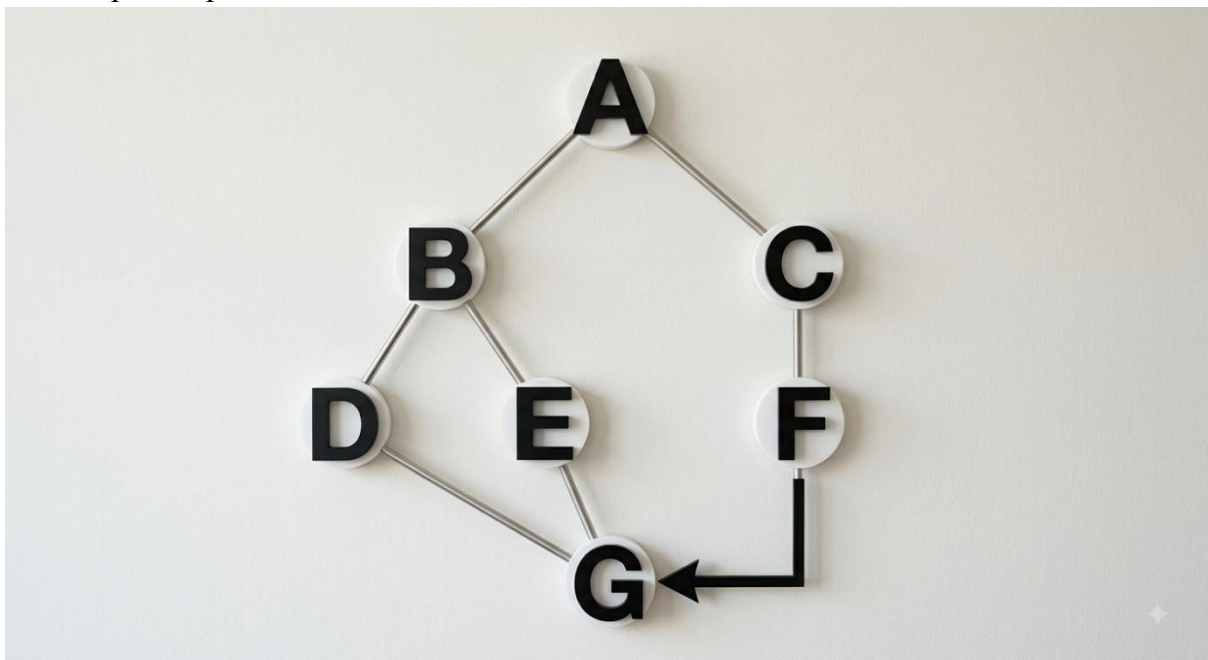
1. Problem Statement

Greedy Best-First Search is used to find a path from a **source node** to a **goal node** by selecting the node that appears closest to the goal.

2. Problem Analysis

- **Initial state:** A
- **Goal state:** G
- **Search method:** Greedy Best-First Search
- **Heuristic:** $h(n)$ = estimated distance from node n to goal G.
-

3. Example Graph



Example heuristic values:

Node	$h(n)$
A	6
B	5
C	3
D	4
E	2
F	1
G	0

4. Algorithm

Greedy Best-First Search chooses the node having the **smallest $h(n)$** .

5. Pseudocode

GreedyBestFirstSearch(start, goal)

1. Put start into OPEN.
2. Set CLOSED = empty.
3. While OPEN is not empty:
 - a. Select node n with minimum $h(n)$.
 - b. Remove n from OPEN.
 - c. If $n = \text{goal}$:
 return path.
 - d. Add n to CLOSED.
 - e. Add unvisited successors of n to OPEN.
4. Return failure if OPEN becomes empty.

6. Node Expansion Order

Starting from A:

A

↓

Compare B(5) and C(3)

↓

Choose C

↓

Choose F(1)

↓

Choose G(0)

Therefore:

Expansion order:

$A \rightarrow C \rightarrow F \rightarrow G$

Path:

$A \rightarrow C \rightarrow F \rightarrow G$

7. Why Greedy Best-First Search?

It is suitable because it uses the heuristic value to move toward the goal quickly. It generally explores fewer nodes than uninformed searches when a good heuristic is available.

8. Complexity

- **Time:** $O(b^m)$
- **Space:** $O(b^m)$

where b is branching factor and m is maximum depth.

9. Advantages

- Simple to implement.
- Uses heuristic information.
- Often reaches the goal quickly.

10. Limitation

It **does not always find the shortest path**, because it considers only $h(n)$ and ignores the cost already travelled.

Conclusion

Greedy Best-First Search is effective when reaching the goal quickly is more important than guaranteeing the shortest path. The document specifically asks for the heuristic, expansion order, and justification.

Q2. A* Search Algorithm

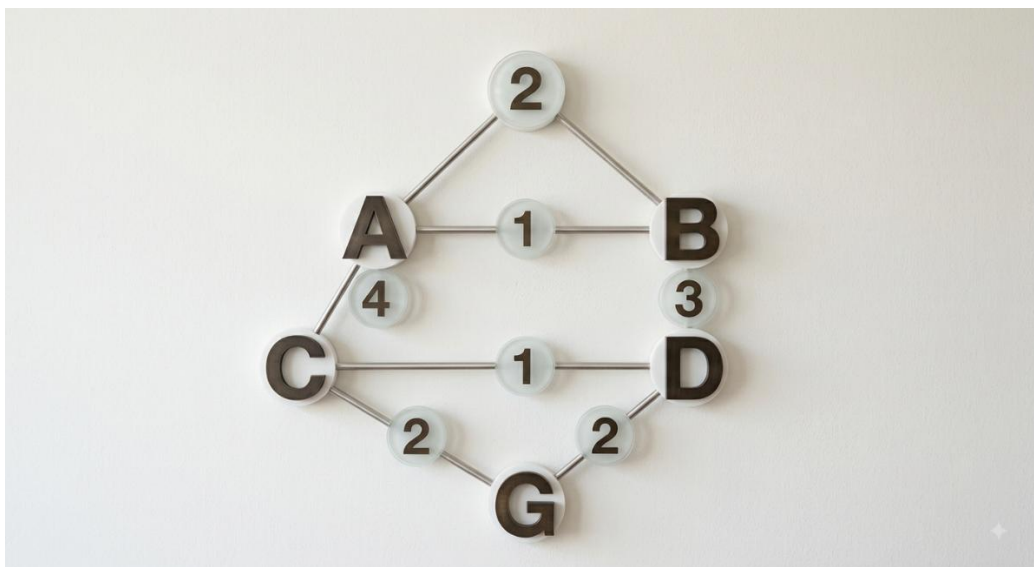
1. Problem Statement

A delivery robot must travel from a warehouse to a customer through a weighted road network. A* Search finds the **optimal path** using both actual cost and estimated cost.

2. Problem Analysis

- **Initial state:** A — Warehouse
- **Goal state:** G — Customer
- **Algorithm:** A*
- **Actual cost:** $g(n)$
- **Heuristic cost:** $h(n)$
- **Total cost:** $f(n)=g(n)+h(n)$

3. Example Graph



Example heuristic values:

Starting at A:

Node	$g(n)$	$h(n)$	$f(n)=g+h$
A	0	6	6
B	2	5	7
C	4	4	8
D	5	2	7
G	7	0	7

The algorithm selects the node with the lowest $f(n)$.

Example optimal path:

$A \rightarrow B \rightarrow D \rightarrow G$

Total cost:

$A \rightarrow B = 2$

$B \rightarrow D = 3$

$D \rightarrow G = 2$

Total = $2 + 3 + 2 = 7$

5. Pseudocode

AStar(start, goal)

1. Put start in OPEN.
2. Set $g(\text{start}) = 0$.
3. While OPEN is not empty:
 - a. Select node n with minimum $f(n)$.
 - b. If $n = \text{goal}$:
return path.
 - c. Move n from OPEN to CLOSED.
 - d. For each successor:
calculate $g(n)$
calculate $h(n)$
calculate $f(n) = g(n) + h(n)$
 - e. Add/update successor in OPEN.
4. Return failure.

6. Why A* Gives an Optimal Solution

A* considers both:

$g(n)$ = cost already travelled

$h(n)$ = estimated remaining cost

Therefore:

$f(n) = g(n) + h(n)$

If the heuristic is **admissible**, meaning it never overestimates the actual remaining cost, A* can guarantee an optimal solution.

7. Complexity

- **Time:** $O(b^d)$ in the general case
- **Space:** $O(b^d)$

8. Advantages

- Finds the shortest/optimal path with a suitable heuristic.
- More efficient than uninformed search in many problems.
- Uses both path cost and heuristic.

Conclusion

A* is suitable for the delivery robot because it considers the actual travelling cost and the estimated distance to the customer.

Q3. Examination Timetable Using CSP

1. Problem Statement

A university needs to create an examination timetable so that **conflicting subjects are not scheduled at the same time**.

The document specifically asks to identify **variables, domains, constraints**, and explain **MRV and Forward Checking**.

2. Problem Analysis

This problem can be represented as a **Constraint Satisfaction Problem (CSP)**.

3. Variables

Suppose the subjects are:

A = Mathematics

B = Physics

C = Computer Science

D = English

E = Electronics

Each subject is a variable.

4. Domains

Suppose there are three examination slots:

$D = \{\text{Slot 1, Slot 2, Slot 3}\}$

Therefore:

$A \in \{1,2,3\}$

$B \in \{1,2,3\}$

$C \in \{1,2,3\}$

$D \in \{1,2,3\}$

$E \in \{1,2,3\}$

5. Constraints

If two subjects have common students, they cannot be at the same time.

For example:

$A \neq B$

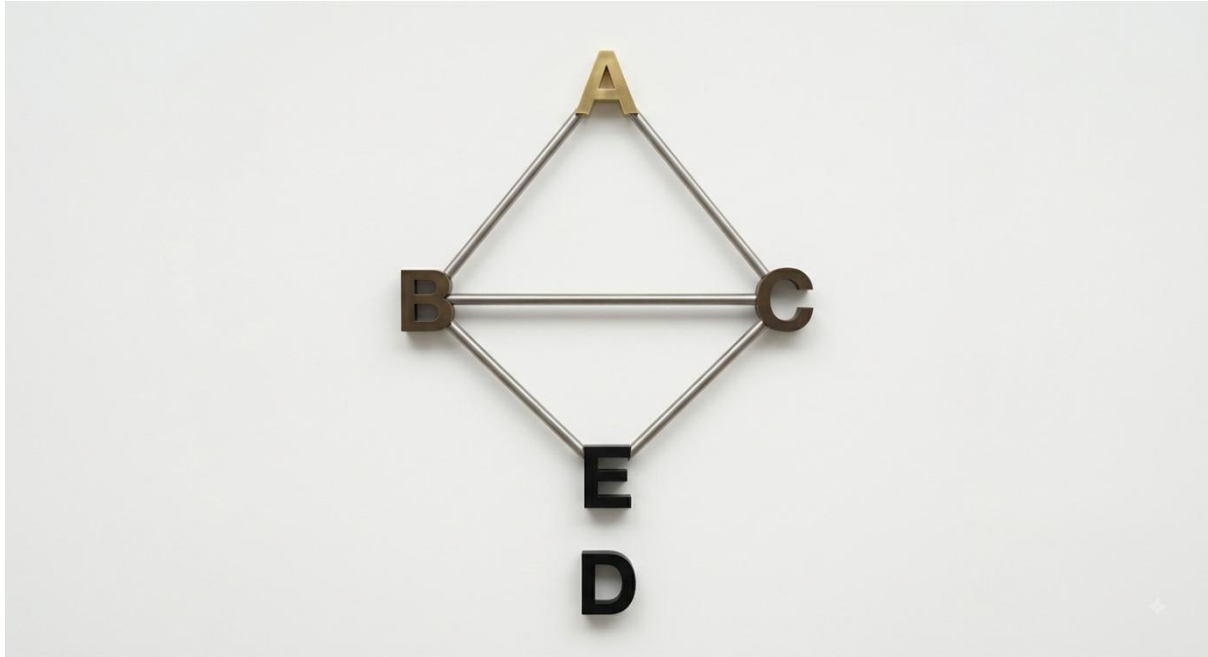
$A \neq C$

$B \neq C$

$B \neq E$

$C \neq E$

Constraint graph:



Here, connected subjects cannot use the same examination slot.

6. MRV

MRV = Minimum Remaining Values

MRV selects the variable having the **smallest number of available values**.

Example:

$A \rightarrow \{1,2,3\}$

$B \rightarrow \{1,2\}$

$C \rightarrow \{1\}$

$D \rightarrow \{1,2,3\}$

C has only one possible value.

Therefore:

Choose C first.

This reduces the search space.

7. Forward Checking

After assigning a value to a variable, Forward Checking removes that value from the domains of its neighboring variables.

Example:

C = Slot 1

If B and E conflict with C:

B = {2,3}

E = {2,3}

Slot 1 is removed from their domains.

8. CSP Algorithm

CSP-Solve()

1. Select an unassigned subject using MRV.
2. Select an available examination slot.
3. Assign the slot.
4. Apply Forward Checking.
5. If any domain becomes empty:
 backtrack.
6. Continue until all subjects are assigned.
7. Return the timetable.

9. Example Timetable

Subject	Slot
Mathematics	1
Physics	2
Computer Science	3
English	1
Electronics	2

No conflicting subjects are placed in the same slot.

10. Complexity

For n subjects and d possible slots:

- **Worst-case time:** $O(d^n)$
- **Space:** $O(n)$ for the assignment, excluding domain storage.

MRV and Forward Checking reduce unnecessary search.

11. Advantages

- Avoids timetable conflicts.
- MRV reduces search.
- Forward Checking detects conflicts early.

Conclusion

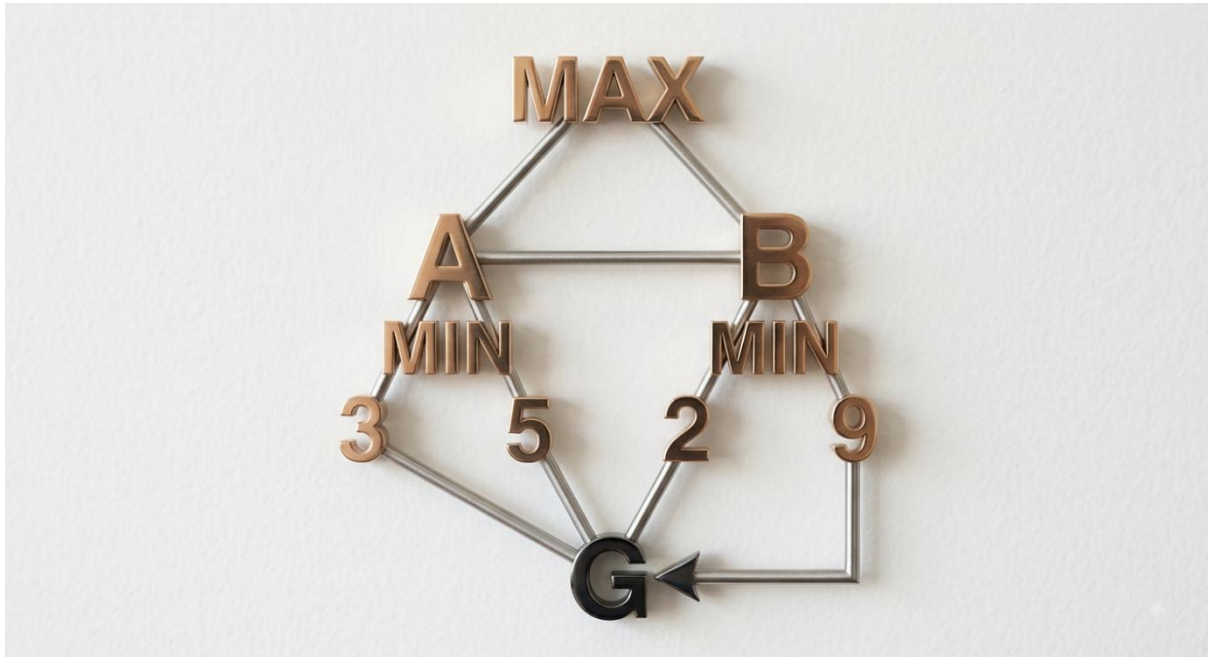
CSP is appropriate because the timetable contains variables, possible values, and restrictions. MRV chooses the most constrained subject first, while Forward Checking prevents invalid assignments early.

Q4. Minimax and Alpha-Beta Pruning

1. Problem Statement

The Minimax algorithm is used in a two-player game to determine the best move for the **MAX player**. Alpha-Beta Pruning improves Minimax by avoiding unnecessary node evaluations.

2. Example Game Tree



3. Minimax Calculation

For node A:

$$A = \text{MIN}(3, 5)$$

$$A = 3$$

For node B:

$$B = \text{MIN}(2, 9)$$

$$B = 2$$

At the MAX level:

$$\text{MAX} = \text{MAX}(3, 2)$$

$$= 3$$

Therefore, MAX chooses:

A

4. Minimax Pseudocode

Minimax(node, depth, maximizingPlayer)

1. If node is terminal:
return utility value.

2. If maximizingPlayer:

best = $-\infty$

for each child:

value = Minimax(child, depth+1, FALSE)

best = max(best, value)

return best

3. Otherwise:

best = $+\infty$

for each child:

value = Minimax(child, depth+1, TRUE)

best = min(best, value)

return best

5. Alpha-Beta Pruning

Two values are maintained:

α = best value found for MAX

β = best value found for MIN

Pruning condition:

if $\alpha \geq \beta$

prune remaining nodes

6. Example

Evaluate A first:

A = MIN(3,5) = 3

Therefore at MAX:

$\alpha = 3$

Now evaluate B.

First child of B:

2

Since B is a MIN node:

$\beta = 2$

Now:

$\alpha = 3$

$\beta = 2$

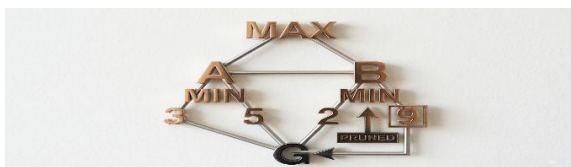
Since:

$\alpha \geq \beta$

$3 \geq 2$

the remaining node 9 is pruned.

So:



7. Node Comparison

Without Alpha-Beta:

Nodes evaluated = 7

With Alpha-Beta:

Nodes evaluated = 6

So one node is avoided in this example.

8. Complexity

Minimax:

Time = $O(b^d)$

Space = $O(bd)$

Alpha-Beta:

Worst case = $O(b^d)$

Best case = $O(b^{(d/2)})$

9. Advantages

- Minimax gives the best decision assuming optimal opponent play.
- Alpha-Beta reduces unnecessary calculations.
- It produces the **same decision** as Minimax but can evaluate fewer nodes.

Conclusion

For the example, MAX selects **A**, with a value of **3**. Alpha-Beta pruning avoids evaluating the unnecessary node 9. The actual document asks to apply this procedure to the **given game tree**; since that tree is not present in the extracted document, the tree above is an illustrative example.

Q5. Warehouse Robot — Online Search

1. Problem Statement

A warehouse robot operates in a dynamic environment. Obstacles can suddenly appear, so the robot must continuously observe the environment and change its path.

2. Problem Analysis

- **Initial state:** Robot's current position
- **Goal:** Destination
- **Environment:** Dynamic
- **Problem:** Unexpected obstacles
- **Suitable technique:** Online Search Agent

3. Why Online Search?

Traditional informed search normally assumes that the environment is known.

But here:

Obstacle appears

↓
 Robot detects it
 ↓
 Update environment
 ↓
 Recalculate path
 ↓
 Continue toward goal
 Therefore, **Online Search** is suitable.

4. Algorithm

OnlineSearch()

1. Start from current position.
2. Sense the environment.
3. Check the next possible movement.
4. If the path is free:
 move to the next position.
5. If an obstacle is detected:
 update the environment.
6. Select another possible path.
7. Continue until the goal is reached.

5. Example

Initial environment:

S → → → → G

Robot starts moving:

S → → R → G

Suddenly an obstacle appears:

S → → R X G

↑
 obstacle

Robot detects the obstacle and changes direction:

→ → G
 /

S → R

The robot does not need to know the entire environment beforehand.

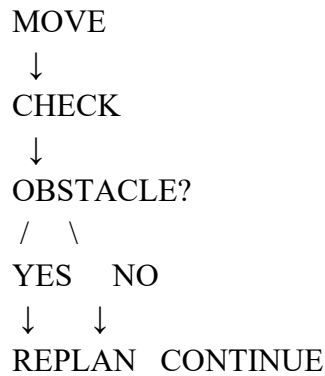
6. Adaptation to Changes

The robot repeatedly:

SENSE

↓
 PLAN

↓



If the environment changes, the robot updates its knowledge and searches for a new path.

7. Comparison

Feature	Online Search	Informed Search
Environment	Partially/initially unknown	Usually known
Dynamic obstacles	Handles well	Less suitable if map changes
Replanning	Yes	Usually requires new search
Information	Obtained while moving	Available before search
Adaptability	High	Lower
Example	Warehouse robot	Fixed road navigation

8. Complexity

The complexity depends on the search strategy and number of states discovered during execution.

In general:

- Time: Depends on explored states and replanning.
- Space: Depends on the states stored by the agent.

9. Advantages

- Works in unknown environments.
- Adapts to obstacles.
- Can replan during movement.
- Useful for real-world robots.

10. Limitations

- May take a longer path.
- Replanning requires additional computation.
- The robot may make decisions without complete information.

Conclusion

Online Search is the better choice for a warehouse robot because obstacles may appear unexpectedly. The robot can sense, update, replan, and continue toward the destination.